A Synapse Matrix That Never Grows

Notes on building Synaptic Memory Lab, and what it does and doesn't tell you about Dragon Hatchling

Every Transformer you've deployed has the same structural tax: the longer the conversation, the bigger the KV cache. Double the context, roughly double the memory and the per-token cost of attending to it. That's not a bug anyone is going to patch away — it's what "attend to every previous token" costs by construction. A model that instead kept a *fixed-size* state, updated it as new information arrived, and still reasoned correctly, would be solving a genuinely different problem, not just a faster version of the same one. That's the premise Pathway's Dragon Hatchling (BDH) architecture is built around, and it's the premise I set out to make someone feel in under two minutes, rather than just read about.

I didn't build a small BDH. I built something much dumber and, I'd argue, more useful for a first fifteen minutes with the idea: an 8-symbol vocabulary, one 8×8 matrix of synapse weights, and two rules — a Hebbian write and a sparse recurrent read. No gradients, no training loop, nothing hidden. You can watch every number in the whole system change. The project is called Synaptic Memory Lab, and the rest of this post is about why that specific toy, rather than a scaled-down neural net, and what it's actually evidence for versus what it's just an illustration of.

The claim, stated so it can be wrong

Before writing a line of the demo, I wrote down one falsifiable sentence, because a "concept demo" that can't fail isn't teaching anything:

> *A fixed-size recurrent state, updated by local Hebbian synaptic plasticity, can encode a novel abstract rule from a handful of demonstrations and then apply it through repeated latent refinement — without allocating a new memory slot for every token, and without ever emitting a verbal chain of thought.*

Every design decision in the lab exists to let a learner break that sentence, not just watch it succeed. The task that carries the whole demo is a two-hop chain: you demonstrate α → γ and separately γ → θ, then query α. At one latent step, the system reads its own matrix once and answers γ — wrong, and visibly so, because ground truth (θ) sits right next to it. Move the latent-refinement slider to two steps, change nothing else, and the same fixed 8×8 matrix now answers θ. Nothing was added to the model's memory to make that happen. It just got read twice.

That's the entire mechanism in one sentence, and it's also the entire mechanism of the sentence I set out to test. If a learner can get the system to answer wrong on purpose — by under-demonstrating, or by leaving latent steps at one — the claim is falsifiable, which was the bar.

Why the toy has to be this small

The design brief for this track is explicit about "visible state": shrink the model until the learner can see every variable that matters. I took that literally. An 8×8 matrix has 64 numbers. You can put all 64 on screen as a data table, and I did — there's a toggle for it next to the graph, because a graph alone isn't a legitimate accessibility fallback and because seeing the raw numbers is occasionally the point.

The three controls that exist — plasticity rate, number of demonstrations, latent-refinement steps — each map to exactly one line in the update rule. There's no decorative fourth slider. Demonstrations write into the matrix (`W[x][y] += η`, then everything decays by a small factor and clips at zero); latent steps read the matrix repeatedly (`a ← top-1(ReLU(a · W))`, run k times). That's genuinely all of it. A learner who reads those two lines has read the entire model.

The honest cost of that legibility: the toy has no representation-learning story at all. Each of the eight symbols is its own dimension. That buys total transparency and buys away any ability to generalize past what's literally been demonstrated — a limitation I'll come back to, because it's the most important thing the demo *doesn't* claim.

Where this sits next to the rest of the fixed-memory literature

BDH is not the only recent architecture trying to escape the growing-cache tax, and the interesting part of doing this research honestly was working out where it actually sits relative to the others, rather than treating it as the only idea in the space.

Retentive Network (Sun et al., 2023, arXiv:2307.08621) gets you an O(1) recurrent inference mode by replacing softmax attention with a decay-weighted retention mechanism — the "memory" is a fixed-size matrix that gets exponentially discounted and updated every step, which is structurally close to what my toy does with its decay term, except RetNet's decay and update are learned end-to-end rather than a hand-set Hebbian rule.

DeltaNet (Yang et al., 2024, arXiv:2406.06484) is the closest published relative to the write rule in this lab. Instead of just adding a new key-value outer product into the state (which is what plain linear attention, and my toy, mostly do), DeltaNet's update *removes* the old value associated with a key before writing the new one — a delta rule, in the classic Widrow-Hoff sense — and the paper's whole contribution is a hardware-efficient way to parallelize that removal-then-write over long sequences. Their 1.3B model, trained on 100B tokens, beat Mamba and gated-linear-attention baselines on perplexity and zero-shot downstream tasks. My toy's plain additive Hebbian write is the simpler, weaker cousin of DeltaNet's delta rule — it can overwrite by decay, but it can't surgically correct a specific stale association the way theirs can.

Titans (Behrouz, Zhong & Mirrokni, Google Research, 2024, arXiv:2501.00663) takes the fixed-size-state idea somewhere more aggressive: a deep neural memory module that updates *its own weights* during the forward pass, learning what to memorize and what to forget at test time, and reportedly scales to context windows over 2 million tokens. That's a genuinely different mechanism from Hebbian writes — it's a learned, gradient-based test-time update rather than a fixed local rule — but the motivating problem is identical: don't let the cache grow, keep useful information in a state that gets rewritten, not appended to.

Lined up, the pattern is: everyone agrees the growing cache is the thing to kill; nobody agrees on the update rule. RetNet learns a decay. DeltaNet learns a targeted correction. Titans learns an entire memory network on the fly. BDH's answer, per its own paper, is to go a level more literal than any of them: don't approximate a brain-like update, use one — Hebbian synaptic writes over a sparse, non-negative, scale-free graph of neuron-like units, framed explicitly as biologically-motivated rather than only computationally convenient.

What BDH itself actually reports

Dragon Hatchling (arXiv:2509.26507, Kosowski, Uznański, Chorowski, Stamirowska & Bartoszkiewicz, Pathway, submitted September 2025) describes a "scale-free biologically inspired network of n locally-interacting neuron particles," in which synapses accumulate short-term memory through Hebbian updates on spiking-neuron-style activity, activations are sparse and strictly non-negative, and the resulting connectivity graph is heavy-tailed rather than uniform — a small number of highly-connected hub synapses, many weakly-connected ones, the same kind of degree distribution you'd cite for a biological neural network rather than a dense weight matrix. The paper reports the architecture matching GPT-2-class Transformer performance across a 10M-to-1B parameter range on language and translation tasks, and — despite being specified as local graph dynamics — admits a GPU-friendly reformulation (BDH-GPU) built from low-rank, ReLU-based linear-attention-style operations, which is what makes it trainable at those scales at all rather than a pure simulation exercise. The project's own repository (MIT-licensed, github.com/pathwaycom/bdh) additionally reports 97.4% accuracy on the Sudoku-Extreme benchmark, a constraint-satisfaction task that's a reasonable stress test for exactly the kind of iterative, non-verbal reasoning this lab's toy model gestures at with its two-latent-step trick.

The successor system, BDH-CQ (arXiv:2608.09888, Engdahl, Kosowski, Chorowski, Stamirowska, Uznański, Jiang, Phadke, Kinas & Zhong, August 2026), is where the "in-context learning without a written chain of thought" half of my claim gets its most direct real-world analogue: demonstrations shown at inference time continuously update the model's recurrent memory, and a query is then resolved through iterative computation in a high-dimensional latent space, with no intermediate reasoning ever verbalized into tokens. The reported evidence is specific rather than aspirational: a 150M-parameter BDH-CQ model reaches 29.5% pass@2 on ARC-AGI-1 at a computed inference cost of $0.0007 per task, which the paper frames as moving the cost–accuracy Pareto frontier for that benchmark rather than just adding another point near the existing one. The problem statement for this track also notes that the BDH-CQ paper explicitly relates its contextual-memory mechanism to attention, fast-weight memory, and linear-attention formulations of association — with the special case where state accumulates *additively* per demonstration, which is precisely the operation this lab's Hebbian write performs, just without BDH-CQ's learned architecture around it.

I want to be exact about what I did and didn't do with those two papers: I read the abstracts, the project's own explainer page, and the repository documentation directly — not a summary of a summary — and every specific number above (10M–1B parameters, 97.4% Sudoku accuracy, 29.5% pass@2, $0.0007/task) is quoted or closely paraphrased from those primary sources, not estimated. I did not run BDH or BDH-CQ myself, and the toy model in this lab is not a compressed

version of either. It's a separate, much simpler system built to demonstrate one of the same *properties* — fixed-size state, evidence accumulated by local writes, reasoning through repeated reads rather than emitted tokens — using a mechanism (top-1 sparsification, symbol-identity representation) that is my own simplification, not theirs.

What the toy actually can't do, and why that's the point

The most common failure mode I'd expect a sharp reviewer to hit is asking the sandbox to generalize. Pick the "Arbitrary Lookup" task, query a symbol whose pairing was never demonstrated, and the system correctly says so: no confident association, at any demonstration count. That's not a bug I'm apologizing for. A model whose neurons *are* symbol identities has no shared structure between symbols to generalize across — it can only ever tell you what it's been directly shown. Real associative-memory and fast-weight architectures — DeltaNet's delta-corrected keys, Titans' learned memory network — get their generalization ability from *distributed* representations that BDH itself also uses (via its expansion representation, which this toy does not reproduce). Trading that away was the price of a matrix small enough to put on screen in full. I think it's a fair trade for a first explanation of the mechanism, and a bad trade if someone mistook this lab for evidence about how well BDH itself generalizes. It isn't.

The second limitation is more subtle: more latent-refinement steps only help when the missing association was already demonstrated. Set demonstrations to just the first hop of the two-hop task and crank latent steps up regardless — the system still returns no answer, because there's nothing in the matrix for the second read to find. Repetition compounds evidence that exists; it doesn't invent evidence that doesn't. That distinction is, I'd argue, the actual content of "reasoning without a chain of thought" — it's not reasoning from nothing, it's reasoning by re-reading a state that was honestly built from what it was shown.

Try it, then decide if it holds up

The guided walkthrough gets to the two-hop flip in under ninety seconds. The sandbox lets you run any of three tasks, any query symbol, any combination of plasticity, demonstration count, and latent steps — including both failure modes above, on purpose. If the one-sentence claim at the top of this post doesn't survive you actually trying to break it, that's a real result, not a demo bug; tell me what broke it.

Primary sources cited above:

- Kosowski, Uznański, Chorowski, Stamirowska & Bartoszkiewicz, *The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain*, arXiv:2509.26507 (2025)
- Engdahl, Kosowski, Chorowski, Stamirowska, Uznański, Jiang, Phadke, Kinas & Zhong, *BDH-CQ: In-Context Learning with Recurrent Latent Reasoning*, arXiv:2608.09888 (2026)
- Sun, Dong, Huang, Ma, Xia, Xue, Wang & Wei, *Retentive Network: A Successor to Transformer for Large Language Models*, arXiv:2307.08621 (2023)

- Yang, Wang, Zhang, Shen & Kim, *Parallelizing Linear Transformers with the Delta Rule over Sequence Length*, arXiv:2406.06484 (2024), NeurIPS 2024
- Behrouz, Zhong & Mirrokni, *Titans: Learning to Memorize at Test Time*, arXiv:2501.00663 (2024)
- Pathway, BDH: The Dragon Hatchling architecture, explained
- pathwaycom/bdh — official implementation, MIT license